Open in Colab
(https://colab.research.google.com/github/mohsin-riad/Epileptic-Seizure-Recognition/blob/main/main.ipynb)

Epileptic Seizure Recognition

###Import Required Libraries

In [1]:
```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import LSTM
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
import warnings
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
import lime
from lime import lime_tabular
warnings.filterwarnings('ignore')
```

```
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:516: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  _np_qint8 = np.dtype([("qint8", np.int8, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:517: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  _np_quint8 = np.dtype([("quint8", np.uint8, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:518: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  _np_qint16 = np.dtype([("qint16", np.int16, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:519: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  _np_quint16 = np.dtype([("quint16", np.uint16, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:520: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  _np_qint32 = np.dtype([("qint32", np.int32, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\framework\dtypes.py:525: FutureWarni
ng: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (ty
pe, (1,)) / '(1,)type'.
  np_resource = np.dtype([("resource", np.ubyte, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:541: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  _np_qint8 = np.dtype([("qint8", np.int8, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:542: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  _np_quint8 = np.dtype([("quint8", np.uint8, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:543: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  _np_qint16 = np.dtype([("qint16", np.int16, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:544: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  _np_quint16 = np.dtype([("quint16", np.uint16, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:545: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  _np_qint32 = np.dtype([("qint32", np.int32, 1)])
C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:550: Futu
reWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood
as (type, (1,)) / '(1,)type'.
  np_resource = np.dtype([("resource", np.ubyte, 1)])
```

Dataframe Enquiry

In [2]:
```
df = pd.read_csv("Dataset/Epileptic Seizure Recognition.csv")
df
```

Out[2]:

	Unnamed	X1	X2	X3	X4	X5	X6	X7	X8	X9	...	X170	X171	X172	X173	X174	X175	X176	X177	X178	y
0	X21.V1.791	135	190	229	223	192	125	55	-9	-33	...	-17	-15	-31	-77	-103	-127	-116	-83	-51	4
1	X15.V1.924	386	382	356	331	320	315	307	272	244	...	164	150	146	152	157	156	154	143	129	1
2	X8.V1.1	-32	-39	-47	-37	-32	-36	-57	-73	-85	...	57	64	48	19	-12	-30	-35	-35	-36	5
3	X16.V1.60	-105	-101	-96	-92	-89	-95	-102	-100	-87	...	-82	-81	-80	-77	-85	-77	-72	-69	-65	5
4	X20.V1.54	-9	-65	-98	-102	-78	-48	-16	0	-21	...	4	2	-12	-32	-41	-65	-83	-89	-73	5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11495	X22.V1.114	-22	-22	-23	-26	-36	-42	-45	-42	-45	...	15	16	12	5	-1	-18	-37	-47	-48	2
11496	X19.V1.354	-47	-11	28	77	141	211	246	240	193	...	-65	-33	-7	14	27	48	77	117	170	1
11497	X8.V1.28	14	6	-13	-16	10	26	27	-9	4	...	-65	-48	-61	-62	-67	-30	-2	-1	-8	5
11498	X10.V1.932	-40	-25	-9	-12	-2	12	7	19	22	...	121	135	148	143	116	86	68	59	55	3
11499	X16.V1.210	29	41	57	72	74	62	54	43	31	...	-59	-25	-4	2	5	4	-2	2	20	4

11500 rows × 180 columns

Feature Label extraction

In [3]:
```
X = df.iloc[:,1:-1]
y = df.iloc[:,-1:]
```

Feature

In [4]:
```
X
```

Out[4]:

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	...	X169	X170	X171	X172	X173	X174	X175	X176	X177	X178
0	135	190	229	223	192	125	55	-9	-33	-38	...	8	-17	-15	-31	-77	-103	-127	-116	-83	-51
1	386	382	356	331	320	315	307	272	244	232	...	168	164	150	146	152	157	156	154	143	129
2	-32	-39	-47	-37	-32	-36	-57	-73	-85	-94	...	29	57	64	48	19	-12	-30	-35	-35	-36
3	-105	-101	-96	-92	-89	-95	-102	-100	-87	-79	...	-80	-82	-81	-80	-77	-85	-77	-72	-69	-65
4	-9	-65	-98	-102	-78	-48	-16	0	-21	-59	...	10	4	2	-12	-32	-41	-65	-83	-89	-73
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11495	-22	-22	-23	-26	-36	-42	-45	-42	-45	-49	...	20	15	16	12	5	-1	-18	-37	-47	-48
11496	-47	-11	28	77	141	211	246	240	193	136	...	-94	-65	-33	-7	14	27	48	77	117	170
11497	14	6	-13	-16	10	26	27	-9	4	14	...	-42	-65	-48	-61	-62	-67	-30	-2	-1	-8
11498	-40	-25	-9	-12	-2	12	7	19	22	29	...	114	121	135	148	143	116	86	68	59	55
11499	29	41	57	72	74	62	54	43	31	23	...	-94	-59	-25	-4	2	5	4	-2	2	20

11500 rows × 178 columns

Label

In [5]:
```
y
```

Out[5]:

	y
0	4
1	1
2	5
3	5
4	5
...	...
11495	2
11496	1
11497	5
11498	3
11499	4

11500 rows × 1 columns

Feature Engineering

Multi label to binary label conversion

In [6]:
```
def toBinary(x):
    if x != 1: return 0;
    else: return 1;
```

In [7]:
```
y = y['y'].apply(toBinary)
y = pd.DataFrame(data=y)
y
```

Out[7]:

	y
0	0
1	1
2	0
3	0
4	0
...	...
11495	0
11496	1
11497	0
11498	0
11499	0

11500 rows × 1 columns

Feature Scaling

In [8]:
```
# scaler = StandardScaler()
# X = scaler.fit_transform(X)
```

In [9]:
```
X
```

Out[9]:

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	...	X169	X170	X171	X172	X173	X174	X175	X176	X177	X178
0	135	190	229	223	192	125	55	-9	-33	-38	...	8	-17	-15	-31	-77	-103	-127	-116	-83	-51
1	386	382	356	331	320	315	307	272	244	232	...	168	164	150	146	152	157	156	154	143	129
2	-32	-39	-47	-37	-32	-36	-57	-73	-85	-94	...	29	57	64	48	19	-12	-30	-35	-35	-36
3	-105	-101	-96	-92	-89	-95	-102	-100	-87	-79	...	-80	-82	-81	-80	-77	-85	-77	-72	-69	-65
4	-9	-65	-98	-102	-78	-48	-16	0	-21	-59	...	10	4	2	-12	-32	-41	-65	-83	-89	-73
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11495	-22	-22	-23	-26	-36	-42	-45	-42	-45	-49	...	20	15	16	12	5	-1	-18	-37	-47	-48
11496	-47	-11	28	77	141	211	246	240	193	136	...	-94	-65	-33	-7	14	27	48	77	117	170
11497	14	6	-13	-16	10	26	27	-9	4	14	...	-42	-65	-48	-61	-62	-67	-30	-2	-1	-8
11498	-40	-25	-9	-12	-2	12	7	19	22	29	...	114	121	135	148	143	116	86	68	59	55
11499	29	41	57	72	74	62	54	43	31	23	...	-94	-59	-25	-4	2	5	4	-2	2	20

11500 rows × 178 columns

In [10]:
```
y
```

Out[10]:

	y
0	0
1	1
2	0
3	0
4	0
...	...
11495	0
11496	1
11497	0
11498	0
11499	0

11500 rows × 1 columns

Splitting Data (Train-Test)

In [11]:
```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.3)
```

In [12]:
```
x_test.iloc[1]
```

Out[12]:
```
X1        5
X2        3
X3        9
X4       16
X5       17
        ...
X174    119
X175    126
X176    132
X177    133
X178    136
Name: 1909, Length: 178, dtype: int64
```

Building Models (classical ML)

Logistic Regression

Training data accuracy evaluation

In [13]:
```
clf = LogisticRegression() #initializing logistic regression
clf.fit(x_train, y_train) #training the model with train data(input, output)
acc_log_reg = clf.score(x_train, y_train) * 100
print(round(acc_log_reg,2), "%")
```

```
67.52 %
```

Test Data accuracy evaluation

In [14]:
```
y_pred_log_reg = clf.predict(x_test)
acc_log_reg2 = round(clf.score(x_test, y_test) * 100, 2)
print(acc_log_reg2, "%")
```

```
63.94 %
```

Model Report

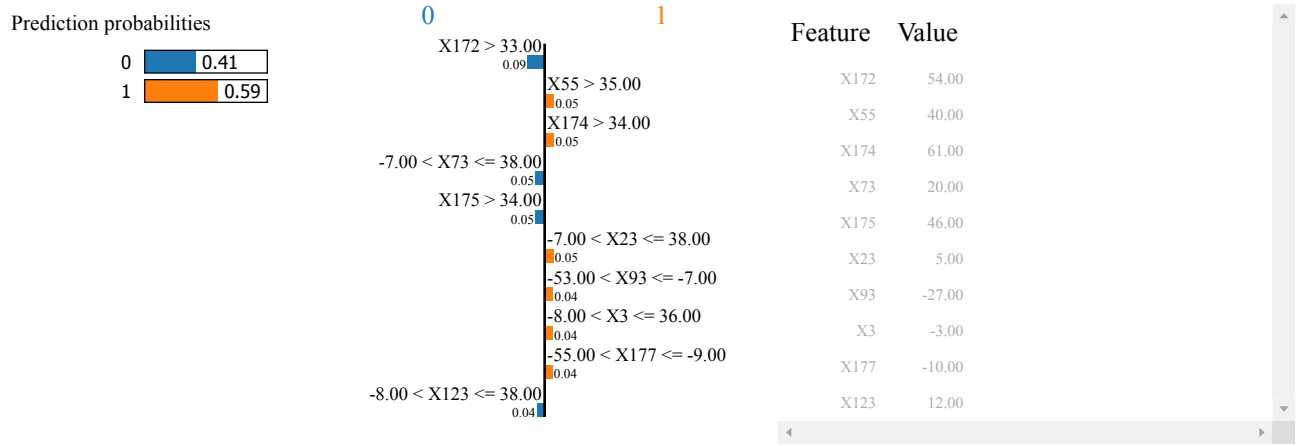
In [15]:
```
predictions = clf.predict(x_test)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.83      0.70      0.76      2767
           1       0.25      0.41      0.31       683

    accuracy                           0.64      3450
   macro avg       0.54      0.55      0.53      3450
weighted avg       0.71      0.64      0.67      3450
```

Model interpretation (LIME)

In [16]:
```
explainer = lime_tabular.LimeTabularExplainer(
    training_data=np.array(x_train),
    feature_names=x_train.columns,
    class_names=[0, 1],
    mode='classification'
)
```

In [17]:
```
exp = explainer.explain_instance(
    data_row=x_test.iloc[0],
    predict_fn=clf.predict_proba
)

exp.show_in_notebook(show_table=True)
```



SVM

Training data accuracy evaluation

In [18]:
```
clf = SVC(probability=True) #initializing svm classifier
clf.fit(x_train, y_train) #training the model with train data(input, output)
acc_svc1 = clf.score(x_train, y_train) * 100
print(round(acc_svc1,2), '%')
```

```
98.31 %
```

Test Data accuracy evaluation

In [19]:
```
y_pred_svc = clf.predict(x_test)
acc_svc2 = round(clf.score(x_test, y_test) * 100, 2)
print(acc_svc2, "%")
```

```
97.33 %
```

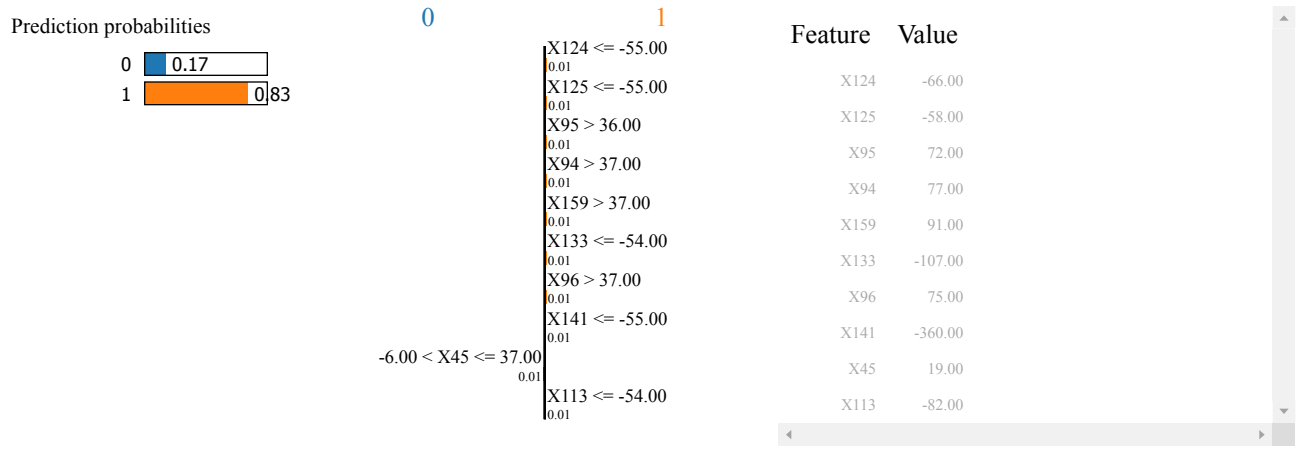
Model Report

In [20]:
```
predictions = clf.predict(x_test)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98      2767
           1       0.97      0.89      0.93       683

    accuracy                           0.97      3450
   macro avg       0.97      0.94      0.96      3450
weighted avg       0.97      0.97      0.97      3450
```

Model interpretation (LIME)

In [21]:
```
exp = explainer.explain_instance(
    data_row=x_test.iloc[2222],
    predict_fn=clf.predict_proba
)

exp.show_in_notebook(show_table=True)
```



Linear SVM

Train Data accuracy evaluation

In [22]:
```
clf = LinearSVC() #initializing svm classifier
# clf = SVC(kernel='linear',probability=True)
clf.fit(x_train, y_train) #training the model with train data(input, output)
acc_linear_svc1 = clf.score(x_train, y_train) * 100
print(round(acc_linear_svc1,2), '%')
```

```
83.33 %
```

Test Data accuracy evaluation

In [23]:
```
y_pred_linear_svc = clf.predict(x_test)
acc_linear_svc2 = round(clf.score(x_test, y_test) * 100, 2)
print(acc_linear_svc2, "%")
```

```
83.88 %
```

Model Report

In [24]:
```
predictions = clf.predict(x_test)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.83      1.00      0.91      2767
           1       0.98      0.19      0.32       683

    accuracy                           0.84      3450
   macro avg       0.91      0.59      0.61      3450
weighted avg       0.86      0.84      0.79      3450
```

KNN

Train Data accuracy evaluation

In [25]:
```
clf = KNeighborsClassifier() #initializing svm classifier
clf.fit(x_train, y_train) #training the model with train data(input, output)
acc_knn1 = clf.score(x_train, y_train) * 100
print(round(acc_knn1,2), '%')
```

```
93.42 %
```

Test Data accuracy evaluation

In [26]:
```
y_pred_knn = clf.predict(x_test)
acc_knn2 = round(clf.score(x_test, y_test) * 100, 2)
print(acc_knn2, "%")
```

```
92.52 %
```

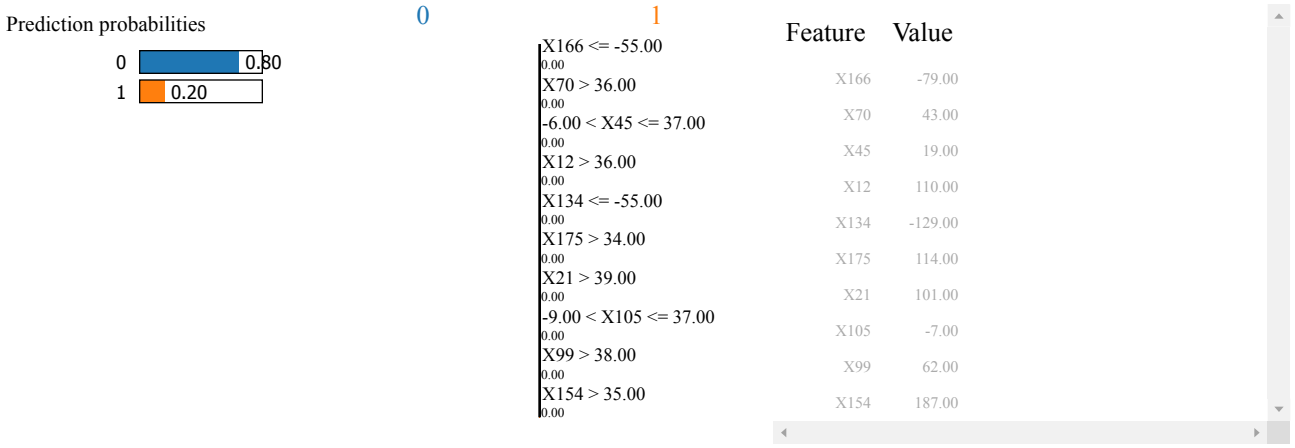
Model Report

In [27]:
```
predictions = clf.predict(x_test)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.92      1.00      0.96      2767
           1       0.99      0.63      0.77       683

    accuracy                           0.93      3450
   macro avg       0.95      0.81      0.86      3450
weighted avg       0.93      0.93      0.92      3450
```

Model interpretation (LIME)

In [28]:
```
exp = explainer.explain_instance(
    data_row=x_test.iloc[2222],
    predict_fn=clf.predict_proba
)

exp.show_in_notebook(show_table=True)
```



RNN (Neural Network)

feature engineering for RNN

In [29]:
```
y = to_categorical(y)
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.2)
X_train = x_train
X_test = x_test

scaler = StandardScaler()

x_train = scaler.fit_transform(x_train)
x_train = np.reshape(x_train, (x_train.shape[0],1,X.shape[1]))

x_test = scaler.fit_transform(x_test)
x_test = np.reshape(x_test, (x_test.shape[0],1,X.shape[1]))
```

In [30]:
```
print(type(x_train))
print(type(x_test))
print(type(X_train))
print(type(X_test))
print(type(y_train))
print(type(y_test))
```

```
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.frame.DataFrame'>
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
```

LSTM / C-LSTM

In [31]:
```
tf.keras.backend.clear_session()

model = Sequential()
model.add(LSTM(64, input_shape=(1,178),activation="relu",return_sequences=True))
model.add(LSTM(32,activation="sigmoid"))
model.add(Dense(2, activation='softmax'))
model.compile(loss = 'categorical_crossentropy', optimizer = "adam", metrics = ['accuracy'])
model.summary()
```

```
WARNING:tensorflow:From C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\ops\init_op
s.py:1251: calling VarianceScaling.__init__ (from tensorflow.python.ops.init_ops) with dtype is deprecated and will be removed
in a future version.
Instructions for updating:
Call initializer instance with the dtype argument instead of passing it to the constructor
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm (LSTM)                  (None, 1, 64)             62208
_________________________________________________________________
lstm_1 (LSTM)                (None, 32)                12416
_________________________________________________________________
dense (Dense)                (None, 2)                 66
=================================================================
Total params: 74,690
Trainable params: 74,690
Non-trainable params: 0
_________________________________________________________________
```

Fitting Model

In [32]:
```
history = model.fit(x_train, y_train, epochs = 10)
```

```
WARNING:tensorflow:From C:\Users\Ram\PycharmProjects\BrainTumorPrediction\venv\lib\site-packages\tensorflow\python\ops\math_grad.py:1250: add_dispatch_support.<locals>.wrapper (from tensorflow.python.ops.array_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
Epoch 1/10
9200/9200 [==============================] - 3s 290us/sample - loss: 0.3705 - acc: 0.8409
Epoch 2/10
9200/9200 [==============================] - 1s 161us/sample - loss: 0.1138 - acc: 0.9690
Epoch 3/10
9200/9200 [==============================] - 2s 168us/sample - loss: 0.0724 - acc: 0.9789
Epoch 4/10
9200/9200 [==============================] - 2s 165us/sample - loss: 0.0519 - acc: 0.9849
Epoch 5/10
9200/9200 [==============================] - 2s 166us/sample - loss: 0.0384 - acc: 0.9884
Epoch 6/10
9200/9200 [==============================] - 2s 167us/sample - loss: 0.0301 - acc: 0.9916
Epoch 7/10
9200/9200 [==============================] - 2s 168us/sample - loss: 0.0233 - acc: 0.9940
Epoch 8/10
9200/9200 [==============================] - 2s 167us/sample - loss: 0.0170 - acc: 0.9964
Epoch 9/10
9200/9200 [==============================] - 2s 170us/sample - loss: 0.0146 - acc: 0.9968
Epoch 10/10
9200/9200 [==============================] - 2s 181us/sample - loss: 0.0108 - acc: 0.9972
```

Accuracy Evaluation

Train data

In [33]:
```
scoreTrain, accTrain = model.evaluate(x_train, y_train)
print(round(accTrain*100, 2), '%')
```

```
9200/9200 [==============================] - 1s 71us/sample - loss: 0.0083 - acc: 0.9986
99.86 %
```

Test Data

In [34]:
```
scoreTest, accTest = model.evaluate(x_test, y_test)
print(round(accTest*100, 2), '%')
```

```
2300/2300 [==============================] - 0s 61us/sample - loss: 0.0906 - acc: 0.9722
97.22 %
```

Individual check

In [35]:
```
print(x_test[22,:])
print(y_test[22,:])
```

```
[[-0.07446214  0.07128929  0.05372041 -0.06631827 -0.24486493 -0.39446194
  -0.39188137 -0.30273872 -0.34879336 -0.30380005 -0.31541503 -0.2643251
  -0.23938116 -0.23351554 -0.40445245 -0.61130076 -0.57020769 -0.38756613
  -0.09754049  0.03111769 -0.03926682 -0.21587334 -0.50556093 -0.66181625
  -0.6466788  -0.55190379 -0.34875113 -0.14728714 -0.07571093 -0.04266876
  -0.1077753  -0.08575841 -0.07859728 -0.05295896  0.05219794  0.14769578
   0.14080175  0.01064429 -0.02775468  0.15096332  0.3465276   0.64758518
   0.72164297  0.55295049  0.38205167  0.19778072  0.15779182  0.15005101
   0.21145751  0.29076553  0.28844182  0.28134836  0.4190493   0.50069759
   0.6618951   0.89087316  0.92164515  0.94415195  0.77340064  0.40185476
   0.05654391 -0.04881702  0.03598554  0.10023086  0.12983246  0.21360965
   0.20185451  0.20491392  0.24243056  0.30688405  0.4434941   0.33650972
   0.2573559   0.27827694  0.26218451  0.39553916  0.62431543  0.80062747
   0.85828439  0.84831465  0.69480445  0.64743702  0.49489165  0.39422667
   0.1741119   0.02314196  0.01831917  0.06735835  0.13432099  0.24282782
   0.35788789  0.25787605  0.06348311 -0.18759486 -0.31002839 -0.3480336
  -0.2373799   0.0032858   0.19907847  0.35783001  0.40000685  0.29587239
   0.09187775  0.03341185  0.02784181  0.00686663 -0.03434564 -0.19189935
  -0.31512136 -0.39509758 -0.47159799 -0.52536725 -0.55007758 -0.47649879
  -0.12087714  0.31476219  0.62844066  0.54980671  0.17236084 -0.26424993
  -0.56423802 -0.53073741 -0.30454648 -0.1159623  -0.00576978  0.04381666
  -0.01206894  0.11965852  0.24443926  0.32924941  0.30771342  0.19660698
  -0.02291879 -0.16373402 -0.30746335 -0.36611605 -0.46557594 -0.59750153
  -0.47632786 -0.3928694  -0.36515394 -0.3464017  -0.35032632 -0.36217776
  -0.51001839 -0.42386882 -0.24053456 -0.0661188   0.1998016   0.32828364
   0.4733534   0.48142006  0.38779735  0.34738483  0.20645886  0.24982416
   0.22981521  0.12289866  0.059807   -0.03508304 -0.13450089 -0.18593627
  -0.09632038  0.01951323  0.02216153 -0.08520517 -0.15479991 -0.19128443
  -0.13330824 -0.15671464 -0.04396091 -0.01543619  0.04039428 -0.01543004
  -0.04675602 -0.0116306   0.12630483  0.3008071 ]]
[1. 0.]
```

In [36]:
```
scoreTest, accTest = model.evaluate(x_test[[44],:], y_test[[44],:])
print(round(accTest*100, 2), '%')
```

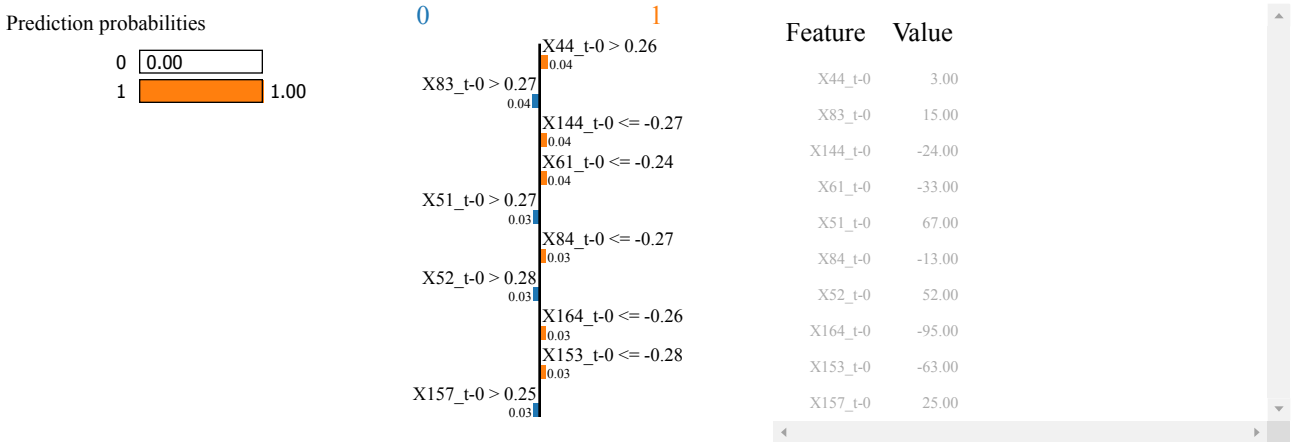
```
1/1 [==============================] - 0s 2ms/sample - loss: 0.0025 - acc: 1.0000
100.0 %
```

In [37]:
```
print(model.predict(x_test[[44],:]))
```

```
[[0.00249478 0.99750525]]
```

Model interpretation (LIME)

In [38]:
```
explainer = lime_tabular.RecurrentTabularExplainer(training_data = x_train,
                                                   feature_names = X_train.columns,
                                                   class_names = [0, 1],
                                                   mode='classification'
                                                   )

exp = explainer.explain_instance(np.array(X_test.iloc[123]), model.predict)
exp.show_in_notebook(show_table=True)
```



In [ ]:
```
```